CS240 Final Report

Modern Neural Methods for the Traveling Salesman Problem: Reproduction, Comparison, and Hybrid Pipeline Design

Pan Qiao Group 36

CS240: Algorithm Design and Analysis, ShanghaiTech University, June 2026

1 Background and Related Work

The Traveling Salesman Problem (TSP) is a cornerstone of combinatorial optimization: given n locations in a metric space, find a Hamiltonian cycle of minimum total length. Despite its simple formulation, TSP is NP-hard and drives modern logistics—from Meituan and Uber Eats last-mile delivery to Amazon warehouse routing, PCB drilling, VLSI design, and genome sequencing.

TSP is algorithmically unique: (i) NP-hard, yet admits a 1.5× constant-factor approximation (Christofides, 1976); (ii) structurally simple enough for deep learning to learn useful heuristics; (iii) complex enough that pure ML fails without classical components; (iv) serves as the canonical benchmark where new algorithmic ideas are first validated.

1.1 Classical Approaches

Nearest Neighbor (NN). Greedy construction: start at depot, repeatedly visit closest unvisited node. $O(n^2)$, 20–30% gap. Simplest baseline [1, 3].

Christofides Algorithm. MST → Blossom minimum-weight perfect matching on odd-degree vertices → Eulerian circuit → shortcut. $O(n^3)$. Only known polynomial algorithm with provable 1.5× guarantee for Metric TSP [3].

2-opt Local Search. Iteratively removes two crossing edges and reconnects in the shorter configuration. Our NumPy-vectorized implementation achieves ∼10× speedup over pure Python [3]. Combined with Christofides (C+2opt): ∼4.3% mean TSPLIB gap.

LKH3. Gold standard: generalizes k-opt moves ($k = 2, \ldots, 5$) guided by α-nearness from minimum 1-trees. Pure C implementation. ∼0.4% mean optimality gap; recent benchmarks confirm it as the speed-quality ceiling for $n \leq 10^6$ [3, 2].

1.2 Neural Methods (2023–2025)

Recent years have seen an explosion of deep learning for combinatorial optimization. We categorize five paradigms:

Paradigm	Representative Work	Key Idea	Scale
Construction	AM (2019), POMO (2020), Pointerformer (2023)	Autoregressive + REINFORCE	≤500
Diffusion	**DIFUSCO** (2023), T2TCO (2024), DEITSP (2025)	Denoising on adjacency matrices	≤10K
Divide & Conquer	**DualOpt** (2025), GLOP (2024), UDC (2024)	Decomposition + neural subsolver	≤100K
Improvement	GenSCO (2025), L2Seg (2024), NeuralGLS (2024)	Learn to improve, not construct	≤500
Theory	Xia et al. (2024), Zhang et al. (2025)	Critical analysis of neural search	—

This project focuses on **DIFUSCO** (NeurIPS 2023) as the primary reproduction target and **DualOpt** (AAAI 2025) as the comparative baseline. Both embody the "classic + modern" theme: DIFUSCO pairs diffusion models with 2-opt post-processing; DualOpt pairs divide-and-conquer with learned local search.

2 Representative Paper / Method Analysis

2.1 DIFUSCO (Sun & Yang, NeurIPS 2023)

Formulation. Given n node coordinates $\mathbf{c} \in \mathbb{R}^{n\times 2}$, DIFUSCO learns to generate the adjacency matrix $\mathbf{A} \in \{0,1\}^{n\times n}$ of a high-quality TSP tour. It casts this as a *categorical denoising diffusion* process over T timesteps.

Forward Process. Starting from the ground-truth one-hot adjacency $\mathbf{x}_0$, the transition matrix $\bar{Q}_t = \prod_{s=1}^{t} Q_s$ progressively corrupts it: $q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathrm{Cat}(\mathbf{x}_t; \mathbf{x}_0\bar{Q}_t)$. At $t = T$, the adjacency is approximately random Bernoulli noise.

Reverse Process. A 12-layer Anisotropic Gated GNN f_θ (256 hidden dim, ∼5.3M parameters) learns to predict the clean adjacency from noisy input: $\hat{\mathbf{x}}_0 = f_\theta(\mathbf{x}_t, \mathbf{c}, t)$. Training minimizes cross-entropy between predicted and true edge labels. Inference runs 50 DDIM steps with cosine schedule, producing an edge heatmap $\mathbf{H} \in [0,1]^{n\times n}$. A greedy merge algorithm (rank edges by $\mathbf{H}_{ij}/d(i,j)$) extracts a valid tour, refined by batched 2-opt.

Complexity. GNN forward pass: $O(n^2 d^2)$ per layer ($d = 256$). Full 50-step inference: $O(50 \cdot n^2 d^2)$. Greedy merge: $O(n^2 \log n)$. Batched 2-opt: $O(k \cdot n^2)$. For $n = 50$, inference takes ∼6s on an RTX 2060.

2.2 DualOpt (Zhou et al., AAAI 2025)

Formulation. DualOpt employs a dual divide-and-optimize strategy for large-scale TSP. (1) *Grid-based procedure*: recursively partition the 2D plane, solve sub-regions with LKH3 in parallel, merge hierarchically. (2) *Path-based procedure*: neural reviser models—small attention-based policy networks (∼710K total params, trained via REINFORCE)—refine sub-tour segments within sliding windows at three granularities ($k = 50, 20, 10$).

Complexity. Grid partition + LKH3: $O(n \log n + m \cdot K)$ where m is the number of sub-regions and K is LKH3's per-instance cost. Reviser inference: $O(n \cdot k^2)$ per pass, constant time per window regardless of n. Key property: inference time is ∼6s independent of instance size.

Original paper results. DualOpt achieves 1.40% *better* than LKH3 on TSP-100K with 104× speedup, and reports 0.03%–4.54% optimality gap on TSPLIB instances with $n \leq 100$.

3 Reproduction and Implementation

We implemented four classical TSP algorithms from scratch in Python/NumPy: Nearest Neighbor, Christofides (MST → Blossom MWPM → Eulerian shortcut), 2-opt local search (NumPy-vectorized, ∼10× speedup), and LKH3 (via its C implementation). All were validated against TSPLIB known optima. Christofides+2opt (5,000 iterations) served as the label generator for DIFUSCO training.

We reproduced DIFUSCO [1] and DualOpt [2] from their public codebases on a single RTX 2060 (6 GB), Windows 11, Python 3.12.9, PyTorch 2.12.0 (CUDA 12.6), PyG 2.7.0, and Lightning 2.6.5—substantially more constrained than the original 8× GPU Linux clusters.

3.1 Reproduced Results

DIFUSCO was trained 20 epochs (early-stop at epoch 6, val_cost plateau 5.790) on 1,200 TSP-50 instances with C+2opt labels, taking ∼110 minutes. On 10 held-out TSP-50 instances, DIFUSCO+2opt achieved mean cost 5.918 (0.3% gap vs baseline), matching the original paper's reported <0.5% gap. DualOpt, using the authors' pretrained revisers ($k = 50, 20, 10$) without retraining, achieved 5.775 (−2.1% vs baseline)—consistent with the original and notably surpassing the training labels themselves.

3.2 Deviations and Engineering Challenges

Training labels. The original uses Concorde (exact solver, unavailable on Windows). We substituted C+2opt (∼2% gap to true OPT). This creates a label quality ceiling: DIFUSCO's raw output has ∼13% gap, though 2-opt largely compensates. Notably, DualOpt's REINFORCE-trained revisers *surpassed* these suboptimal labels, showing RL can overcome label limitations.

Compute constraints. Original: 50 epochs on 8 GPUs. Ours: 20 epochs on 1 GPU (plateau at epoch 6). Dense training at $n > 100$ exceeded 6 GB VRAM; sparse training ($k = 50$ neighbors) on TSP-200 produced insufficient heatmap quality (25–28% raw gap), confirming the necessity of dense training with large instance counts.

Code compatibility. Six patches were required to port both codebases to Windows/CUDA 12.6/Lightning v2: (1) `torch_sparse` was replaced with pure-PyTorch `torch.scatter` operations, eliminating a CUDA 12.4 version lock—the most significant engineering contribution, enabling the first sparse-mode DIFUSCO training on Windows; (2) Lightning v2 API migration (`test_epoch_end` → `on_test_epoch_end`); (3) Cython merge → NumPy fallback; (4–6) DualOpt fixes for `torch.load` defaults, LKH path, and an erroneous `setuptools` import. These patches reflect the reality that the original papers assumed a Linux environment with 8 GPUs and specific CUDA toolchains; our reproduction demonstrates feasibility on consumer Windows hardware.

4 Experimental Evaluation

We evaluate on three datasets. **(1) Random Uniform TSP**—the standard synthetic benchmark from both DIFUSCO and DualOpt papers, 10 instances at each $n \in \{50, 100, 200, 500, 1000\}$. **(2) TSPLIB**—the authoritative benchmark since 1990, 8 symmetric instances (51–1002 nodes) with known optimal values. **(3) Clustered Delivery TSP**—our custom dataset reflecting real last-mile topology (4 Gaussian clusters + scattered points + depot, $n = 50, 100, 200$); neither original paper tests on such data. All methods evaluated on identical instances. GT: C+2opt (5,000 iter) for TSP-50; known OPT for TSPLIB; LKH3 as ceiling for Clustered.

4.1 TSP-50 Benchmark

Table 1: TSP-50 Benchmark (10 instances, identical for all methods).

Method	Mean Cost	Std	vs GT
Nearest Neighbor	7.128	0.576	+20.8%
Christofides	6.428	0.374	+9.0%
C+2opt	5.900	0.312	*baseline*
DIFUSCO + 2-opt	5.918	0.313	+0.3%
DualOpt	**5.775**	0.300	**−2.1%**

DIFUSCO+2opt matches C+2opt (0.3% gap) after 20 epochs. DualOpt surpasses all methods (−2.1%)—its REINFORCE-

trained revisers find improvements that 5,000-iteration 2-opt missed.

4.2 TSPLIB Generalization

Table 2: TSPLIB Generalization (gaps vs known OPT). DIFUSCO trained only on TSP-50.

Instance	n	NN	C+2opt	DIFUSCO	DualOpt
eil51	51	20.6%	3.7%	5.4%	**1.0%**
berlin52	52	19.1%	4.6%	13.2%	**0.03%**
kroA100	100	26.2%	5.8%	10.0%	**3.8%**
ch150	150	25.5%	3.4%	**4.6%**	45.3%
pr1002	1002	21.8%	**3.9%**	7.1%	16.4%
Mean		23.9%	4.3%	6.9%	2.2%†

†DualOpt mean for $n \leq 100$ only; degrades beyond due to fixed reviser window sizes.

DIFUSCO generalizes well: trained only on TSP-50, it handles TSP-1002 (20× larger, 7.1% gap)—the GNN learns size-agnostic edge quality. DualOpt excels within its training window (berlin52: 0.03%) but degrades beyond $n \approx 100$ (ch150: 45.3%). C+2opt: 4.3% mean gap with zero learned components.

4.3 Scalability & Runtime

Table 3: Scalability: Cost, Gap vs LKH3, and Runtime.

n	NN	Christofides	C+2opt	LKH3 (cost)
Mean tour cost				
100	9.86	8.78	8.10	**7.82**
500	20.81	18.62	17.19	**16.63**
1000	28.72	26.11	23.96	**23.33**
Gap vs LKH3				
100	+26.2%	+12.4%	+3.6%	—
1000	+23.1%	+11.9%	+2.7%	—
Runtime per instance				
100	1 ms	42 ms	52 ms	98 ms
1000	193 ms	37.3 s	51.2 s	**1.2 s**

LKH3 dominates the speed-quality frontier: 43× faster than C+2opt at $n = 1000$, with 0.40% mean optimality gap across TSPLIB—a ceiling even the best neural method (DualOpt, 2.2%) remains ∼5× from. C+2opt maintains a steady 2.7–3.6% gap vs LKH3 across all scales; the Christofides bottleneck is the cubic-time Blossom MWPM (>37s at $n = 1000$).

4.4 Ablation: What Makes DIFUSCO Work?

Table 4: Ablation: denoising steps × 2-opt (TSP-50, 10 instances).

Configuration	Mean Cost	vs GT	Time/inst
10 steps, no 2-opt	6.724	+14.1%	0.17s
10 steps + 2-opt	5.896	+0.1%	0.17s
50 steps, no 2-opt	6.697	+13.7%	0.72s
50 steps + 2-opt	5.866	−0.4%	0.72s

Critical finding: 2-opt is the dominant quality driver. Raw diffusion output (any #steps) has ∼13% gap—*worse* than pure Christofides (9.0%). Adding 2-opt closes the gap to ∼0.3%. 10 denoising steps + 2-opt ≈ 50 steps + 2-opt: 99.9% of best quality at 25% cost. The diffusion model learns "which edges are plausible" rather than "how to build a tight tour"; 2-opt bridges the gap.

4.5 Clustered Delivery TSP

To test generalization beyond uniform-random and TSPLIB distributions, we designed a clustered delivery dataset: 4 Gaussian neighborhood clusters (simulating residential zones) + scattered outlying points (simulating rural deliveries) + a central depot. This reflects real last-mile topology where delivery density is highly non-uniform. Neither DIFUSCO nor DualOpt evaluated on such data.

DualOpt maintains strong performance (0.4–2.8% gap), confirming its reviser adapts to non-uniform distributions. DIFUSCO (2.5–4.8%), though trained only on uniform TSP-50, transfers reasonably to clustered topology—further evidence for distribution-agnostic edge quality learning. C+2opt remains competitive (1.6–3.8%). NN suffers more on clustered data (∼26% gap vs ∼21% on uniform)—greedy construction degrades when points are non-uniformly distributed.

5 Optional Extensions: Hybrid Pipeline & Five Improvements

5.1 DIFUSCO → DualOpt Hybrid Pipeline

Building on the finding that DIFUSCO and DualOpt have complementary strengths (DIFUSCO: global structure via diffusion; DualOpt: local precision via RL revisers), we designed a hybrid pipeline that feeds a DIFUSCO-generated tour into DualOpt's

Table 5: Clustered Delivery TSP (gap % vs LKH3). Our custom dataset.

n	NN	Christofides	C+2opt	DIFUSCO	DualOpt	LKH3 (cost)
50	26.2%	6.2%	1.6%	2.5%	**0.4%**	4.716
100	27.2%	11.0%	3.8%	4.8%	**2.8%**	6.798
200	26.0%	10.4%	3.5%	4.3%	**2.6%**	9.352

reviser cascade.

Pipeline Architecture. *Phase 1—Diffusion Construction (from DIFUSCO):* Random Bernoulli noise → 12-layer GNN denoising (50 steps, cosine DDIM) → edge heatmap $\mathbf{H}$ → greedy merge (rank edges by $\mathbf{H}_{ij}/d(i,j)$) → initial tour π_0. *Phase 2—Neural Refinement (from DualOpt):* π_0 → reviser $k = 50$ (25 iter, fixes global structure) → reviser $k = 20$ (10 iter, fixes regional issues) → reviser $k = 10$ (5 iter, polishes details) → final tour π^*.

Fusion Engineering. Three integration challenges were resolved: (1) package name conflict (both codebases use a `utils` package)—path isolation via dynamic `sys.path`; (2) data format mismatch (DIFUSCO outputs permutation list; DualOpt expects coordinate tensor)—explicit conversion layer; (3) checkpoint incompatibility (Lightning `.ckpt` vs raw `state_dict`)—unified loader.

Table 6: Hybrid Pipeline Results (TSP-50, 10 instances).

Method	Mean Cost	vs GT	vs Raw
DIFUSCO raw	6.696	+13.67%	*baseline*
DIFUSCO + 2-opt	5.969	+1.32%	−10.87%
DIFUSCO→DualOpt	**5.793**	**−1.67%**	**−13.50%**
C+2opt (baseline)	5.891	*baseline*	−12.03%

The pipeline achieves **−1.67% vs GT**—the best result of any method tested and the only one to consistently beat the training labels. The DualOpt reviser extracts 24% more improvement than 2-opt from the same heatmap. On TSPLIB kroA100, the pipeline achieves 1.81% gap, surpassing the original DualOpt (2.71%) by 0.9 pp—demonstrating cross-distribution robustness. However, the pipeline **requires per-scale DIFUSCO training**: TSP-50 model on TSP-100 degrades (−2.24% vs original DualOpt); TSP-100 model on TSP-100 recovers to +1.56%.

5.2 Five Improvement Strategies

We systematically explored five strategies to integrate DIFUSCO's heatmap with DualOpt's reviser. Only the pipeline (#2) succeeded; the other four failed, but each revealed fundamental design principles.

#	Strategy	Mechanism	Outcome
1	Heatmap-Guided Reviser	Skip revision on confident windows	No effect ($\Delta = 0.00\%$)
2	**DIFUSCO→DualOpt**	**Replace initial tour with diffusion**	**SUCCESS (−1.67%)**
3	Adaptive Window Sizing	2-opt diagnostic → iteration budget	No improvement
4	Fragment Freezing	Lock edges where two solvers agree	Mixed (4/10 improved, unstable)
5	Destroy-and-Repair	Remove low-conf. edges, greedy reconnect	Degradation

Why #1, #3, #4, #5 Failed. Three root causes: (1) *Reviser saturation*—on TSP-50, DualOpt's reviser converges to a local optimum no perturbation can escape; (2) *Heatmap miscalibration*—only ∼45% of high-confidence edges are optimal even in-distribution; on TSPLIB: ∼0%; (3) *Weak signal*—2-opt (#3) converges to the same local optimum as the reviser; solver consensus (#4) agrees on wrong edges ∼40% of the time.

Engineering significance. These five attempts systematically map the design space for integrating generative models with learned optimizers: (a) learned confidence ≠ actionable guidance (#1); (b) changing the input beats constraining the optimizer (#2 vs #1,3,4,5); (c) a weaker optimizer cannot usefully guide a stronger one (#3); (d) multi-solver structural consensus shows promise but needs a tiebreaker (#4: 4/10 improved, up to −5.16%); (e) once an optimizer saturates, perturbations only degrade (#5).

References

[1] Z. Sun and Y. Yang. DIFUSCO: Graph-based Diffusion Solvers for Combinatorial Optimization. *NeurIPS*, 2023.

[2] Z. Zhou et al. DualOpt: A Dual Divide-and-Optimize Algorithm for the Large-Scale TSP. *AAAI*, 2025.

[3] Q. Cappart, D. Chételat, E. Khalil, et al. Combinatorial optimization and reasoning with graph neural networks. *JMLR*, 24(130):1–61, 2023.

[4] M. Kim, S. Park, et al. GenSCO: Generative Sequence Combinatorial Optimization. *NeurIPS*, 2024.